# CH8 Search-Trees

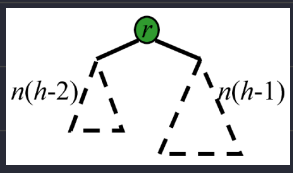
## AVL Trees

### Height-Balance Property

如果一個 Internal 節點 v 的兩個 Children 的高度差 ≤ 1，這樣 v 就是balanced

### Definition of AVL Tree

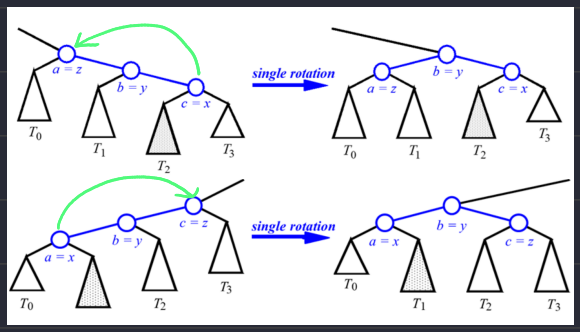
- A binary search tree, which follows right children is greater than the left and each internal node of it are balanced.
- The height of AVL tree $O(n \log n)$



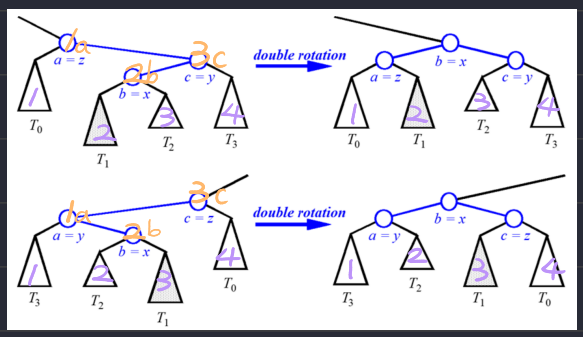

### Insertion $O(\log n)_{Find} + O(1)_{Restructuring} = O(\log n)$

1. Find the correct position for the new node (like BST)
2. Check if the tree satisfied balance property from bottom to top
3. Rotate until all nodes satisfied the balanced property

Single Rotate




Double Rotate




a) 左右順序標示 a, b, c 三個節點，T0 T1 T2 T3 三個子樹

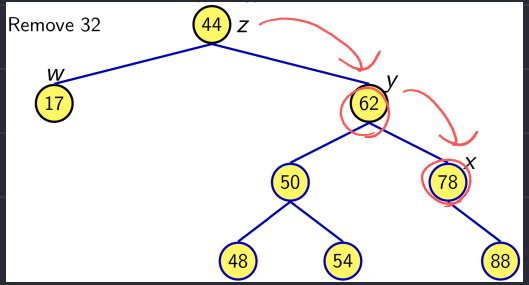
b) 將 b 作為根，a 放左，c 放右

c) 子樹部分依據左右順序插回

p.s. Rotate is also called "trinode restructuring"

### Removal $O(\log n)_{Find} + O(\log n)_{Restructuring} = O(\log n)$

1. Find the correct position for the new node (like BST)
2. Check if the tree satisfied balance property from removal's parent to root
3. First unbalanced node is z, the larger child of z is x, larger child of x is y



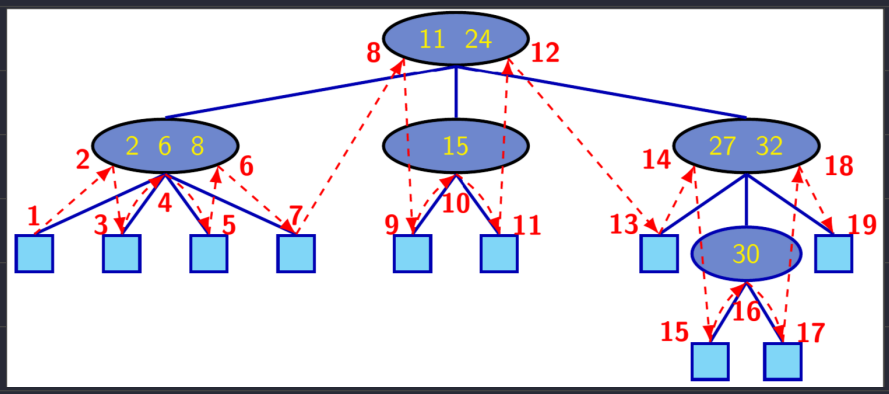
# Multi-Way Search Tree

## Definition

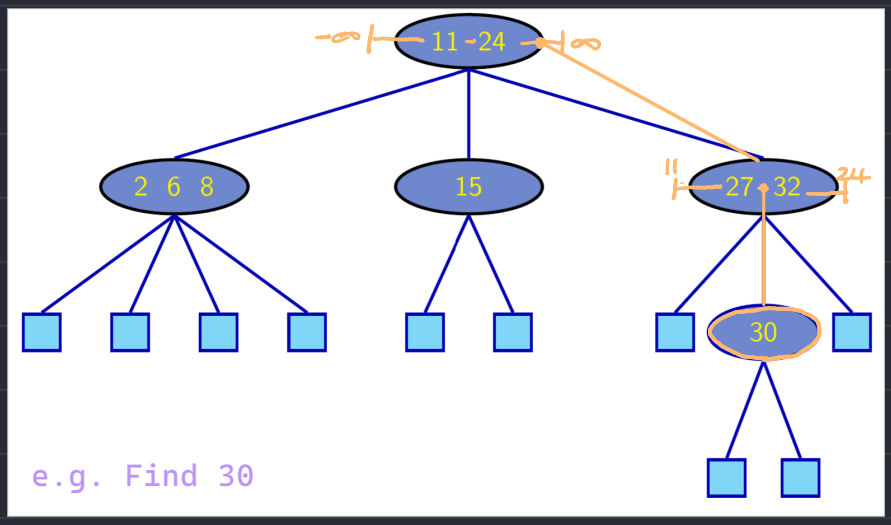
A binary tree that consist of the node which have at least 1 key and 2 chidlren

The numbers of children is not limited by 2

## Inorder Traversal




## Search



## (2, 4) Tree

### B-tree of order m

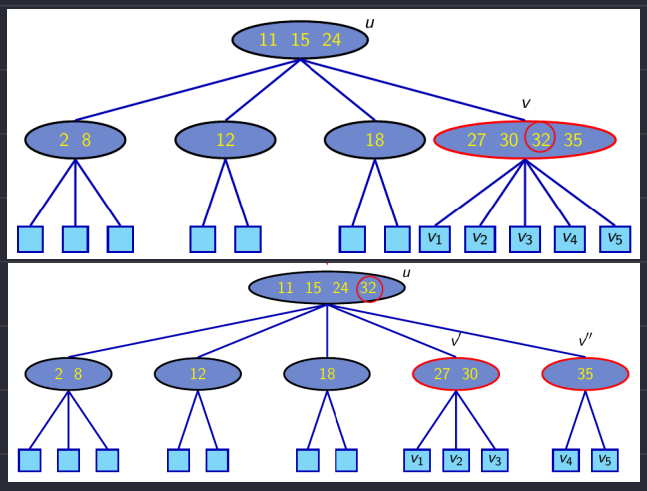
1. The root node has at least two children
2. All nodes other than the root and external nodes have $\lceil \frac{m}{2} \rceil$ to m children
3. All external nodes are at the same level.

### Definition

A kind of multiway search tree with m = 4, so the count of children is 2 ~ 4

### Insertion $\log n$

1. Search to find the proper position to insert
2. Check if overflow happen, if so take the action below:







a. Locate the newly inserted node

b. Move them to the root

c. Overflow might propergate, do the check repeatly

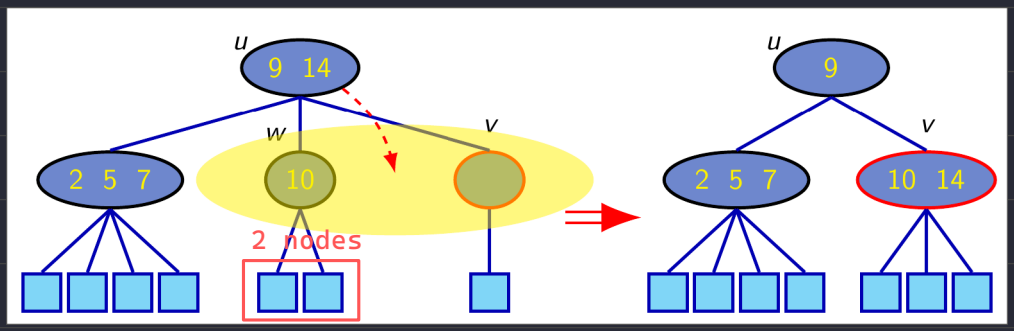
### Removal $\log n$

1. Search to find the proper position to remove
2. Check if underflow happen, if so take the action below:

Underflow

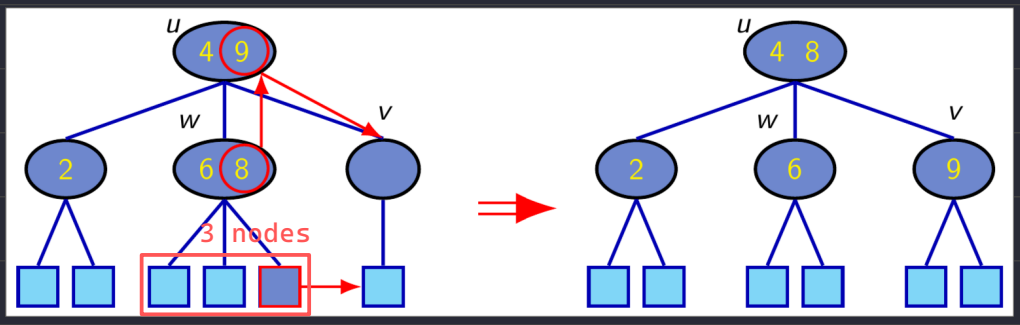
Case 1 (Fusion)

After a fusion, the underflow may propagate to the parent u



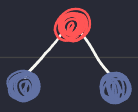

Case 2 (Transfer)

沒有改動 Parent 的 key 數量，因此不會傳遞 underflow



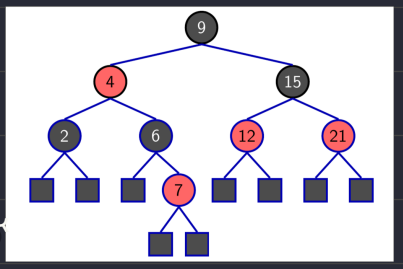
# Red-Black Trees

## Definition



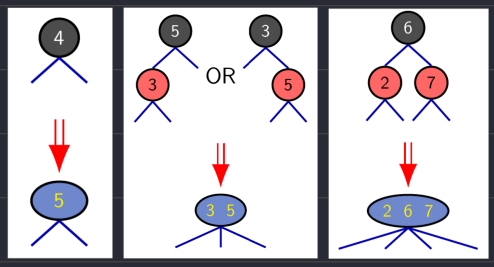
- A kind of binary search tree
- Root Property: the root is black
- External Property: every leaf is black
- Internal Property: the children of a red node are black
- Depth Property: all the leaves have the same black depth

沒有反向規定




## Convert to a (2, 4) tree

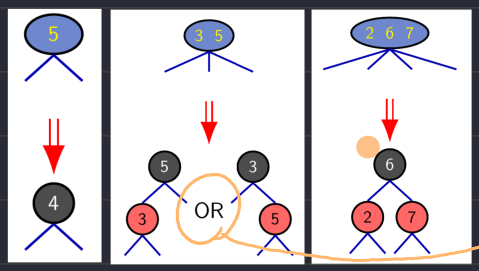
1. Merging every red node into its parent
2. Storing the entry from the red node at its parent




→ 轉換結果是唯一的

## From (2,4) to Red-Black Trees

1. Color each node black first




→ 轉換結果並非唯一

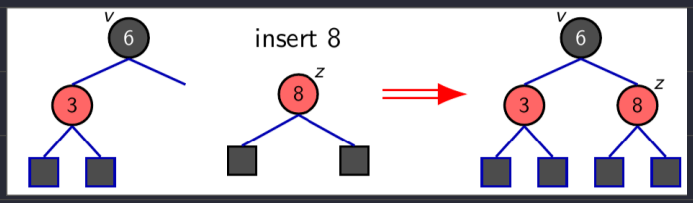
## Height of a Red-Black Tree

The height of a red-black tree is at most twice the height of its associated (2,4) tree, which is O(log n)

Insertion $O(\log n)$

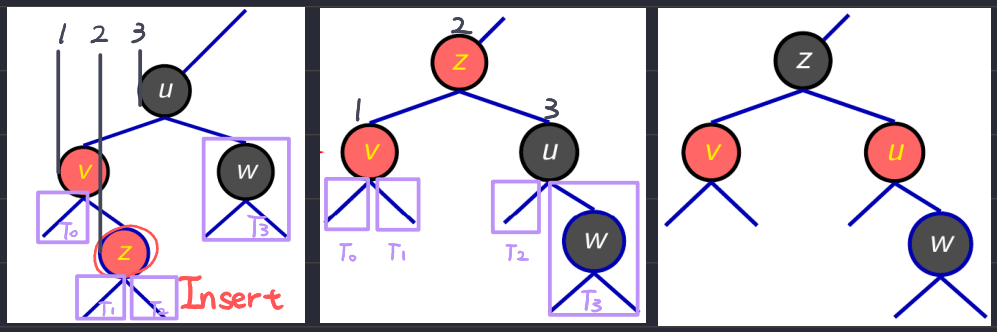
1. 將加入節點塗成紅色，避免影響黑色深度

2a. If the parent v of z is black, it preserve the internal property and we're dor



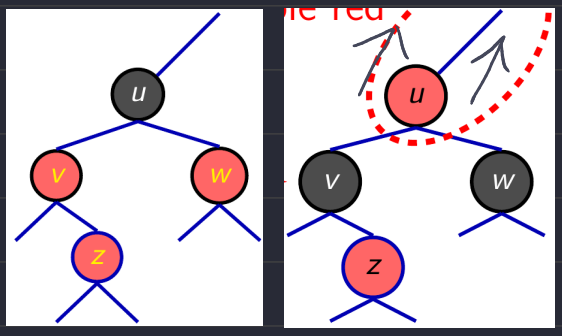

2b. Else (v is red ) we have a double red, restructuring the tree is required.

Case 1 : If z.parent.sibling == black




a) 標記　　b) 排序　　c) 著色

Case 2 : If z.parent.sibling == red




此方法會將 double red 問題往上遷移

a) 著色

Removal $O(\log n)$

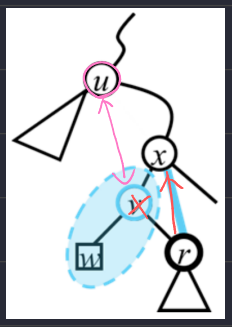
1. Search for the node that wanted to be deleted
2. Start from this node find its Successor or Predecessor

   Successor: 將 Tree 轉成 inorder 後，緊鄰目標節點前後的元素

   e.g. [A,B,C] A is the Successor of B, C is the Predecessor of B

   Way to find in tree: Successor → 右邊子樹的最左

   Predecessor → 左邊子樹的最右



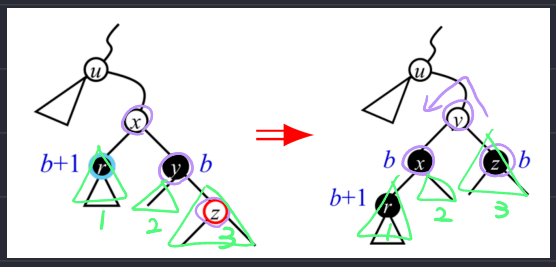

3. Swap these two nodes (方便操作)
4. Color the node we actually deleted black
5. Check if encounter the double black problem

   a. 父節點是紅或黑，且 v 為紅 → 不需要作任何事

   b. 父節點是紅，且 v 為黑 → 補償：將父節點塗黑

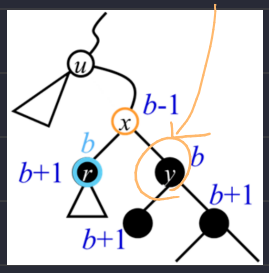
   c. 父節點是黑，且 v 為黑 → 父節點本就是黑色時要承載兩黑 (Double Black)

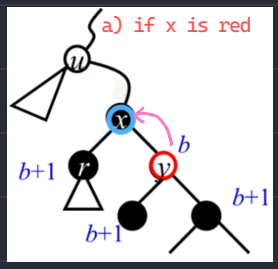
      Case 1 - Restructuring

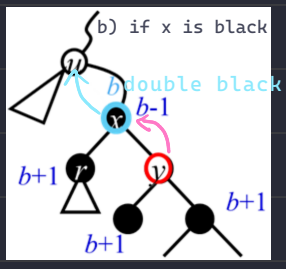



      Case 2 - Recoloring

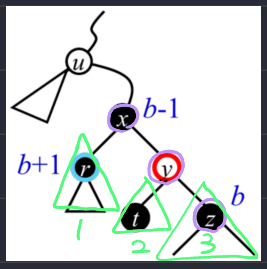
      將對面節點的黑色往上搬移（互換），有可能向上傳遞 double black 問題

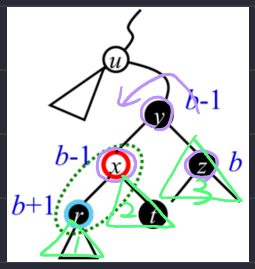



      Case 3 - Adjustment




      對面是紅色的，沒辦法搬 ...

      將 Case 3 的情況轉換成 Case 1, 2

      → 旋轉政策：逆時鐘旋轉

      → 塗色政策：不要動到子樹的黑色深度